Exp:7 Compare the performance of Single Source Shortest Paths using Greedy method when the graph is represented by adjacency matrix and adjacency lists.

AIM:

To implement the Single Source Shortest Path (SSSP) algorithm using the Greedy method (Dijkstra's algorithm) and compare its performance using:

1. Adjacency Matrix
2. Adjacency List

PROCEDURE:

Dijkstra's Algorithm:

1. Read the number of vertices n.
2. Read the **cost matrix** of the graph.
3. Select the **source vertex**.
4. **Initialize**
    1. Set known[v] = 0 for every vertex v.
    2. Set Dv = ∞ for every vertex.
    3. Set Pv = -1 for every vertex.
    4. Set Ds = 0 for the source vertex s.
5. Select the **unknown vertex** v **having the smallest** Dv.
6. **Mark as known**
   Set known[v] = 1.
7. **Update adjacent vertices**
   For every vertex w adjacent to v:
   If known[w] = 0 and
   Dv+cost(v,w)<Dw
   then:
    1. Dw = Dv + cost(v,w)
    2. Pw = v
8. Repeat steps 2–4 until all vertices become known.
9. Display Vertex, Known, Dv, and Pv.
10. If there is no edge place it is 0 and it converts to infinity.

Performance Comparison:

Representation	Time Complexity	Space Complexity
Adjacency Matrix	$O(V^2)$	$O(V^2)$
Adjacency List	$O(V^2 + E)$ in our simple implementation	$O(V + E)$

PROGRAM:

1. Adjacency Matrix:

Adj7.c

```
#include <stdio.h>
#define INF 999
int main()
{
    int cost[10][10];
    int known[10], dv[10], pv[10];
    int n, i, j, v, w, min, source;

    printf("Enter number of vertices: ");
```

```
  scanf("%d", &n);

  printf("Enter cost matrix:\n");
  for(i = 0; i < n; i++)
  {
        for(j = 0; j < n; j++)
        {
           scanf("%d", &cost[i][j]);
           if(cost[i][j] == 0)
               cost[i][j] = INF;
        }
  }

  printf("Enter source vertex: ");
  scanf("%d", &source);

/* Initialization */
  for(i = 0; i < n; i++)
 {
        known[i] = 0;
        dv[i] = cost[source][i];
        pv[i] = source;
 }

dv[source] = 0;
pv[source] = -1;

/* Dijkstra algorithm */
 for(i = 0; i < n; i++)
 {
       min = INF;
       v = -1;
       /* Find unknown vertex with minimum Dv */
       for(j = 0; j < n; j++)
       {
           if(known[j] == 0 && dv[j] < min)
           {
              min = dv[j];
              v = j;
           }
       }
       if(v == -1)
           break;
       known[v] = 1;
       /* Update Dv and Pv */
       for(w = 0; w < n; w++)
       {
           if(known[w] == 0 && dv[v] + cost[v][w] < dv[w])
           {
               dv[w] = dv[v] + cost[v][w];
               pv[w] = v;
           }
       }
  }
  printf("\nVertex\tKnown\tDv\tPv\n");
```

```
    for(i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\n", i, known[i], dv[i], pv[i]);
    }
    return 0;
}
```

OUTPUT :

```
D:\POORNIMA\cprogram>gcc Adj7.c -o Adj7.exe
D:\POORNIMA\cprogram>Adj7.exe
Enter number of vertices: 4
Enter cost matrix:
0 2 0 1
0 0 0 2
3 0 0 0
0 0 1 0
Enter source vertex: 0

Vertex   Known   Dv     Pv
 0        1       0     -1
 1        1       2      0
 2        1       2      3
 3        1       1      0
```

```
D:\POORNIMA\cprogram>gcc Adj6.c -o Adj6.exe

D:\POORNIMA\cprogram>Adj6.exe
Enter number of vertices: 4
Enter cost matrix:
0 2 0 1
0 0 0 2
3 0 0 0
0 0 1 0
Enter source vertex: 0

Vertex  Known   Dv      Pv
0       1       0       -1
1       1       2       0
2       1       2       3
3       1       1       0
```

2. Adjacency List:
Adl7.c

```
#include <stdio.h>
#define MAX 20
#define INF 999

int main()
{
   int adj[MAX][MAX];
   int known[MAX], dv[MAX], pv[MAX];
   int n, e, source;
   int i, j, u, v, w;
   int min, vertex;

   printf("Enter the vertices: ");
```

```
scanf("%d", &n);

/* Initialize graph */
for(i = 0; i < n; i++)
{
   for(j = 0; j < n; j++)
   {
      adj[i][j] = 0;
   }
}

printf("Enter the number of edges: ");
scanf("%d", &e);

printf("Enter the adjacency vertex with edges:\n");

for(i = 0; i < e; i++)
{
   scanf("%d %d %d", &u, &v, &w);

   adj[u][v] = w;
   adj[v][u] = w;   /* For undirected graph */
}

printf("Enter source vertex: ");
scanf("%d", &source);

/* Initialization */
for(i = 0; i < n; i++)
{
   known[i] = 0;
   dv[i] = INF;
   pv[i] = -1;
}

dv[source] = 0;

/* Dijkstra's Algorithm */
for(i = 0; i < n; i++)
{
   min = INF;
   vertex = -1;

   /* Find minimum Dv */
   for(j = 0; j < n; j++)
   {
      if(known[j] == 0 && dv[j] < min)
      {
         min = dv[j];
         vertex = j;
      }
   }

   if(vertex == -1)
      break;
```

```
        known[vertex] = 1;

        /* Update Dv and Pv */
        for(v = 0; v < n; v++)
        {
           if(adj[vertex][v] != 0 && known[v] == 0)
           {
              if(dv[vertex] + adj[vertex][v] < dv[v])
              {
                 dv[v] = dv[vertex] + adj[vertex][v];
                 pv[v] = vertex;
              }
           }
        }
     }

     printf("\nVertex\tKnown\tDv\tPv\n");

     for(i = 0; i < n; i++)
     {
        printf("%d\t%d\t%d\t%d\n", i, known[i], dv[i], pv[i]);
     }

     return 0;
}
```

OUTPUT:

```
D:\POORNIMA\cprogram>gcc Adl7.c -o Adl7.exe
D:\POORNIMA\cprogram>Adl7.exe
Enter the vertices: 4
Enter the number of edges: 5
Enter the adjacency vertex with edges:
0 1 2
0 3 1
1 3 2
3 2 1
2 0 3
Enter source vertex: 0

Vertex   Known   Dv    Pv
 0        1       0     -1
 1        1       2     0
 2        1       2     3
 3        1       1     0
```

```
D:\POORNIMA\cprogram>gcc Ad16.c -o Ad16.exe

D:\POORNIMA\cprogram>Ad16.exe
Enter the vertices: 4
Enter the number of edges: 5
Enter the adjacency vertex with edges:
0 1 2
0 3 1
1 3 2
3 2 1
2 0 3
Enter source vertex: 0

Vertex  Known   Dv      Pv
0       1       0       -1
1       1       2       0
2       1       2       3
3       1       1       0
```

RESULT:

Thus, the Single Source Shortest Path (SSSP) problem using Dijkstra's Greedy method was successfully implemented using both adjacency matrix and adjacency list representations. Both programs produce the same shortest-path distances. The adjacency matrix is simpler and suitable for dense graphs, whereas the adjacency list uses less memory and is better for sparse graphs.

EXP :8 Implement Job Sequencing with deadlines using Greedy strategy.

AIM:

To implement Job Sequencing with Deadlines using the Greedy method and maximize the total profit.

PROCEDURE:

1. Enter the jobs with their deadline and profit.
2. Sort the jobs in decreasing order of profit.
3. Start with the job having the highest profit.
4. Place the job in the latest free slot before its deadline.
5. Repeat until all jobs are checked.
6. Add the profits of the selected jobs.
7. Display the selected jobs and maximum profit.

PROGRAM:

Jsd.c:

```
#include <stdio.h>
int main()
{
   int n, i, j, temp;
   char job[20], tempJob;
   int deadline[20], profit[20];
   int timeslot[20];
   int maxDeadline = 0;
   int totalProfit = 0;

   printf("Enter number of jobs: ");
   scanf("%d", &n);

   printf("Enter Job ID, Deadline and Profit:\n");

   for(i = 0; i < n; i++)
   {
      scanf(" %c %d %d", &job[i], &deadline[i], &profit[i]);

      if(deadline[i] > maxDeadline)
         maxDeadline = deadline[i];
   }

   /* Sort jobs according to profit */
   for(i = 0; i < n - 1; i++)
   {
      for(j = i + 1; j < n; j++)
      {
         if(profit[i] < profit[j])
         {
            temp = profit[i];
            profit[i] = profit[j];
```

```
                profit[j] = temp;

                temp = deadline[i];
                deadline[i] = deadline[j];
                deadline[j] = temp;

                tempJob = job[i];
                job[i] = job[j];
                job[j] = tempJob;
            }
        }
    }

    /* Initialize time slots */
    for(i = 0; i < maxDeadline; i++)
    {
        timeslot[i] = -1;
    }

    /* Schedule jobs */
    for(i = 0; i < n; i++)
    {
        for(j = deadline[i] - 1; j >= 0; j--)
        {
            if(timeslot[j] == -1)
            {
                timeslot[j] = i;
                totalProfit = totalProfit + profit[i];
                break;
            }
        }
    }

    printf("\nTime Slot\tJob\n");

    for(i = 0; i < maxDeadline; i++)
    {
        if(timeslot[i] != -1)
        {
            printf("%d\t\t%c\n", i + 1, job[timeslot[i]]);
        }
    }

    printf("\nMaximum Profit = %d\n", totalProfit);

    return 0;
}
```

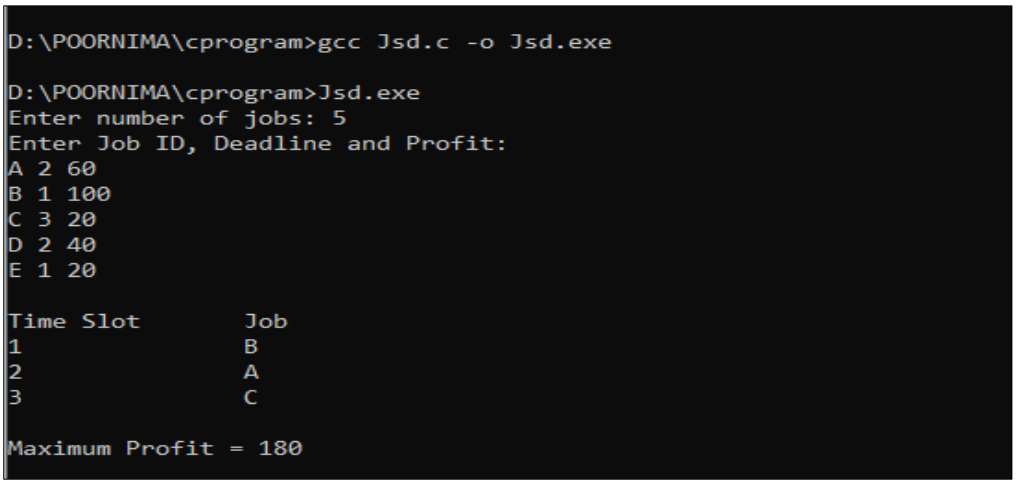
OUTPUT:

```
D:\POORNIMA\cprogram>gcc Jsd.c -o Jsd.exe

D:\POORNIMA\cprogram>Jsd.exe
Enter number of jobs: 5
Enter Job ID, Deadline and Profit:
A 2 60
B 1 100
C 3 20
D 2 40
E 1 20

Time Slot       Job
1               B
2               A
3               C

Maximum Profit = 180
```



RESULT:

Thus, Job Sequencing with Deadlines was successfully implemented using the Greedy strategy in C, and the maximum profit was obtained**.**

EXP :9 WRITE A PROGRAM TO SOLVE 0/1 KNAPSACK PROBLEM USING DYNAMIC PROGRAMMING.

AIM:

To write a C program to solve the 0/1 Knapsack problem using Dynamic Programming.

PROCEDURE:

1. Start the program.
2. Read the number of items and knapsack capacity.
3. Read the weight and profit of each item.
4. Create a Dynamic Programming table `V[i][w]`.
5. Initialize the first row of the table to `0`.
6. For each item and capacity:
   - If the item weight is greater than the capacity, copy the previous value.
   - Otherwise, use the formula:

     V(i,w)=max(V(i-1,w),V(i-1,w-w[i])+p[i]))

7. The value `V[n][W]` gives the maximum profit.
8. Display the maximum profit.
9. Stop the program.

PROGRAM:

knap9.c

```
#include <stdio.h>

#define MAX 100

int max(int a, int b)
{
   return (a > b) ? a : b;
}

int main()
{
   int n, W;
   int w[MAX], p[MAX];
   int V[MAX][MAX];
   int i, j;

   printf("Enter number of items: ");
   scanf("%d", &n);

   printf("Enter capacity of knapsack: ");
   scanf("%d", &W);

   printf("Enter weights of items:\n");
```

```
    for (i = 1; i <= n; i++)
        scanf("%d", &w[i]);

    printf("Enter profits of items:\n");
    for (i = 1; i <= n; i++)
        scanf("%d", &p[i]);

    /* Initialization */
    for (j = 0; j <= W; j++)
        V[0][j] = 0;

    /* Dynamic Programming */
    for (i = 1; i <= n; i++)
    {
        for (j = 0; j <= W; j++)
        {
            if (w[i] > j)
            {
                V[i][j] = V[i - 1][j];
            }
            else
            {
                V[i][j] = max(V[i - 1][j], V[i - 1][j - w[i]] + p[i]);
            }
        }
    }

    printf("\nMaximum Profit = %d\n", V[n][W]);

    /* Display DP table */
    printf("\nDP Table:\n");
    for (i = 0; i <= n; i++)
    {
        for (j = 0; j <= W; j++)
        {
            printf("%4d", V[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

OUTPUT:

D:\>gcc knap9.c -o knap9.exe

D:\>knap9.exe

Enter number of items: 4

Enter capacity of knapsack: 8

Enter weights of items:

2 3 4 5

Enter profits of items:

1 2 5 6

Maximum Profit = 8

DP Table:

0 0 0 0 0 0 0 0 0

0 0 1 1 1 1 1 1 1

0 0 1 2 2 3 3 3 3

0 0 1 2 5 5 6 7 7

0 0 1 2 5 6 6 7 8

```
D:\>gcc knap9.c -o knap9.exe

D:\>knap9.exe
Enter number of items: 4
Enter capacity of knapsack: 8
Enter weights of items:
2 3 4 5
Enter profits of items:
1 2 5 6

Maximum Profit = 8

DP Table:
   0   0   0   0   0   0   0   0   0
   0   0   1   1   1   1   1   1   1
   0   0   1   2   2   3   3   3   3
   0   0   1   2   5   5   6   7   7
   0   0   1   2   5   6   6   7   8
```

RESULT:

Thus, the 0/1 Knapsack problem was successfully solved using Dynamic Programming**,** and the maximum profit was obtained.

EXP :10 IMPLEMENT N-QUEENS PROBLEM USING BACKTRACKING.

AIM:

To implement the N-Queens problem using Backtracking.

PROCEDURE:

1. Start the program.
2. Read the value of N.
3. Place queens row by row.
4. Check whether the queen can be placed safely in the column.
5. If safe, place the queen and move to the next row.
6. If no position is possible, backtrack to the previous row.
7. Continue until all queens are placed.
8. Display the solution.
9. Stop the program.

PROGRAM:

nqueen10.c

```
#include <stdio.h>
#include <stdlib.h>

int n;
int x[20];

int place(int k, int i)
{
   int j;

   for(j = 1; j < k; j++)
   {
      if(x[j] == i || abs(x[j] - i) == abs(j - k))
         return 0;
   }

   return 1;
}

void nqueen(int k)
{
   int i, j;

   for(i = 1; i <= n; i++)
   {
      if(place(k, i))
      {
         x[k] = i;
```

```
            if(k == n)
            {
               printf("\nSolution:\n");

               for(i = 1; i <= n; i++)
               {
                  for(j = 1; j <= n; j++)
                  {
                     if(x[i] == j)
                        printf(" Q ");
                     else
                        printf(" . ");
                  }
                  printf("\n");
               }

               printf("The optimal solution is:");
               for(i = 1; i <= n; i++)
               printf("%d ", x[i]);
               printf("\n");
            }
            else
            {
                  nqueen(k + 1);
             }
          }
       }
    }

int main()
{
   printf("Enter number of queens: ");
   scanf("%d", &n);

   nqueen(1);

   return 0;
}
```

OUTPUT:

D:\>gcc nqueen10.c -o nqueen10.exe

D:\>nqueen10.exe

Enter number of queens:

4

Solution:

. Q . .

. . . Q

Q . . .

. . Q .

The optimal solution is:2 4 1 3

Solution:

. . Q .

Q . . .

. . . Q

. Q . .

The optimal solution is:3 1 4 2

```
D:\>
D:\>gcc nqueen10.c -o nqueen10.exe

D:\>
D:\>nqueen10.exe
Enter number of queens:
4

Solution:
 .  Q  .  .
 .  .  .  Q
 Q  .  .  .
 .  .  Q  .
The optimal solution is:2 4 1 3

Solution:
 .  .  Q  .
 Q  .  .  .
 .  .  .  Q
 .  Q  .  .
The optimal solution is:3 1 4 2

D:\>
```

RESULT:

Thus, the N-Queens problem was successfully implemented using Backtracking and all possible solutions were displayed.

EXP :11 USE BACKTRACKING STRATEGY TO SOLVE 0/1 KNAPSACK PROBLEM.

AIM:

To solve the 0/1 Knapsack problem using Backtracking strategy.

PROCEDURE:

1. Start the program.
2. Read the number of items and knapsack capacity.
3. Read the weights and profits of all items.
4. Consider each item either **selected (1)** or **not selected (0)**.
5. Calculate the total weight and profit.
6. If the total weight is within the capacity, check the profit.
7. Keep track of the maximum profit.
8. Backtrack and try other combinations.
9. Display the maximum profit and selected items.
10. Stop the program.

PROGRAM:

knap11.c

```
#include <stdio.h>
int n, W;
int w[20], p[20];
int x[20], best[20];
int maxProfit = 0;
void knapsack(int i, int weight, int profit)
{
   int j;

   if(i == n)
   {
      if(profit > maxProfit)
      {
         maxProfit = profit;

         for(j = 0; j < n; j++)
            best[j] = x[j];
      }
      return;
   }

   /* Select the item */
   if(weight + w[i] <= W)
   {
      x[i] = 1;
```

```
        knapsack(i + 1, weight + w[i], profit + p[i]);
    }

    /* Do not select the item */
    x[i] = 0;
    knapsack(i + 1, weight, profit);
}

int main()
{
    int I,  totalWeight = 0;

    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter capacity: ");
    scanf("%d", &W);

    printf("Enter weights:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &w[i]);

    printf("Enter profits:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &p[i]);

    knapsack(0, 0, 0);
   printf("\nOptimal Solution: ");
   for(i = 0; i < n; i++)
   {
        printf("%d ", best[i]);
        if(best[i] == 1)
            totalWeight += w[i];
   }

    printf("\nSelected items: ");
    for(i = 0; i < n; i++)
    {
        if(best[i] == 1)
            printf("%d ", i + 1);
    }

    printf("\nMaximum Profit = %d\n", maxProfit);


    return 0;
}
```

OUTPUT:

D:\>gcc knap11.c -o knap11.exe

D:\>knap11.exe

Enter number of items:

4

Enter capacity: 8

Enter weights:

2 3 4 5

Enter profits:

1 2 5 6

Optimal Solution: 0 1 0 1

Selected items: 2 4

Maximum Profit = 8



RESULT:

Thus, the 0/1 Knapsack problem was successfully solved using the Backtracking strategy, and the maximum profit was obtained.

EXP:12 IMPLEMENT TRAVELLING SALES PERSON PROBLEM USING BRANCH AND BOUND APPROACH

AIM:

To implement the Travelling Salesperson Problem using Branch and Bound approach**.**

PROCEDURE:

1. Start the program.
2. Read the number of cities.
3. Read the cost matrix.
4. Start the tour from city 0.
5. Calculate the cost of each possible path.
6. Use Branch and Bound to eliminate paths whose cost cannot give a better solution.
7. Find the minimum cost tour.
8. Display the minimum cost and the optimal path.
9. Stop the program.

PROGRAM:

Tsp12.c

```
#include <stdio.h>
#define INF 9999
int n;
int cost[10][10];
int visited[10];
int path[10], bestPath[10];
int minCost = INF;
void tsp(int city, int count, int totalCost)
{
   int i;

   if(count == n)
   {
      if(cost[city][0] != 0 &&
        totalCost + cost[city][0] < minCost)
      {
         minCost = totalCost + cost[city][0];

         for(i = 0; i < n; i++)
            bestPath[i] = path[i];
      }
      return;
   }

   for(i = 0; i < n; i++)
   {
      if(!visited[i] && cost[city][i] != 0)
      {
```

```
            if(totalCost + cost[city][i] < minCost)
            {
                visited[i] = 1;
                path[count] = i;

                tsp(i, count + 1,
                    totalCost + cost[city][i]);

                visited[i] = 0;
            }
        }
    }
}

int main()
{
    int i, j;

    printf("Enter number of cities: ");
    scanf("%d", &n);

    printf("Enter cost matrix:\n");

    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    for(i = 0; i < n; i++)
        visited[i] = 0;

    visited[0] = 1;
    path[0] = 0;

    tsp(0, 1, 0);

    printf("\nMinimum Cost = %d\n", minCost);

    printf("Optimal Path: ");

    for(i = 0; i < n; i++)
        printf("%d -> ", bestPath[i] + 1);

    printf("1\n");

    return 0;
}
```

OUTPUT:

D:\>gcc Tsp12.c -o Tsp12.exe

D:\>Tsp12.exe

Enter number of cities: 4

Enter cost matrix:

0 10 15 20

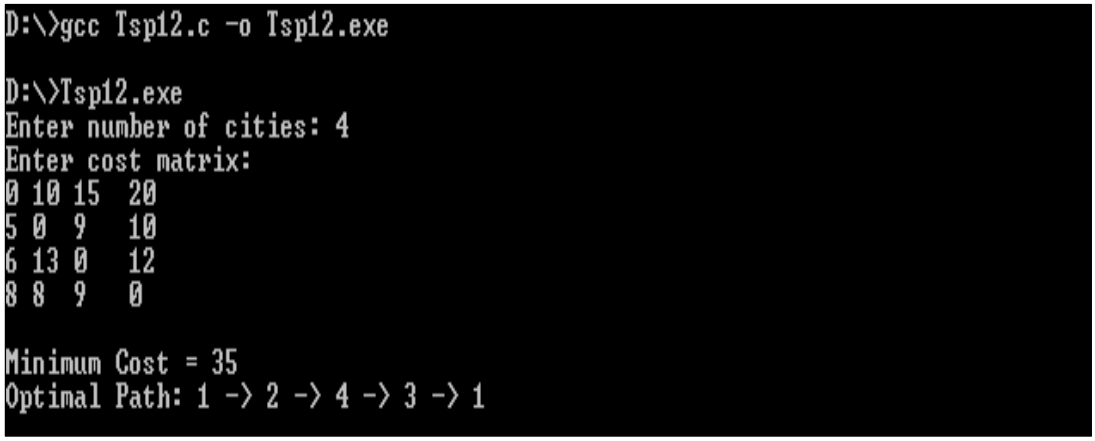
5 0 9   10

6 13 0   12

8 8 9   0

Minimum Cost = 35

Optimal Path: 1 -> 2 -> 4 -> 3 -> 1



RESULT:

Thus, the Travelling Salesperson Problem was successfully implemented using the Branch and Bound approach**,** and the minimum cost tour was obtained.